Malware Basics: Static and Dynamic Analysis

1. Introduction

Malware analysis is the process of examining a suspicious or potentially malicious file to understand its characteristics, functionality, and behavior.

As part of this task, a **benign Python sample** was used to demonstrate basic malware-analysis techniques without executing actual malicious software. Both **static analysis** and **dynamic analysis** were performed in the Kali Linux environment.

The analysis was conducted in a dedicated working directory named `malware-analysis`, with dynamic execution performed inside a `sandbox` directory.

2. Objective

The objectives of this exercise were to:

- Understand the basic malware-analysis workflow.
- Perform static analysis of a sample file.
- Identify the file type and calculate its cryptographic hash.
- Inspect the contents and printable strings of the sample.
- Execute the benign sample in an isolated sandbox directory.
- Observe the behavior of the sample during execution.
- Verify the artifacts created by the sample.
- Understand the difference between static and dynamic malware analysis.

3. Test Environment

Component	Details
Operating System	Kali Linux
Analysis Directory	`~/malware-analysis`
Dynamic Analysis Directory	`~/malware-analysis/sandbox`
Sample	`benign_sample.py`
Sample Type	Python script / ASCII text executable
Analysis Techniques	Static and Dynamic Analysis

The sample used in this exercise was explicitly designed to perform harmless analysis-related activity rather than malicious operations.

4. Malware Analysis Methodology

The analysis was divided into two major phases:

Phase 1 — Static Analysis

The sample was examined without executing it. The following techniques were used:

- File-type identification
- SHA-256 hash calculation
- Source-code inspection
- Printable-string extraction
- File metadata inspection

Phase 2 — Dynamic Analysis

The sample was executed inside a dedicated `sandbox` directory. Its output and filesystem changes were then observed and verified.

5. Static Analysis

5.1 File Type Identification

The following command was used:

```
file benign_sample.py
```

The output identified the file as:

```
benign_sample.py: Python script, ASCII text executable
```

This established that the sample was a Python script represented as ASCII text.

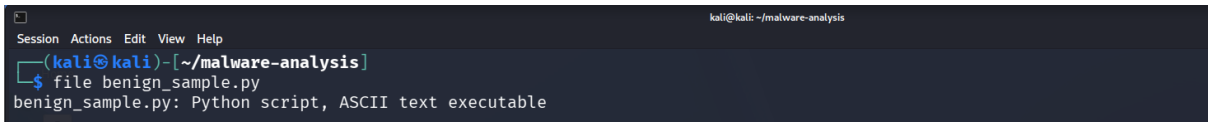

Figure 1: Identification of the benign sample using the `file` command

5.2 SHA-256 Hash Calculation

A SHA-256 cryptographic hash was calculated using:

```
sha256sum benign_sample.py
```

The resulting hash was displayed by the terminal.

The SHA-256 hash provides a unique fingerprint for the analyzed file and can be used to identify the exact sample analyzed during the exercise.

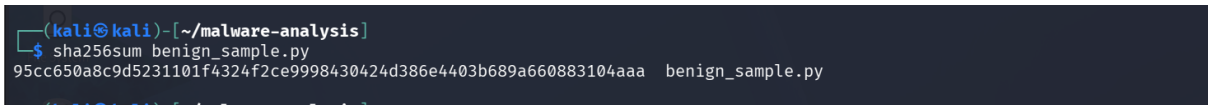

Figure 2: SHA-256 hash calculation of the analysis sample

Note: The complete hash value should be copied directly from the screenshot into the final document if the report requires the exact hash.

5.3 Source-Code Inspection

The contents of the sample were inspected using:

```
cat benign_sample.py
```

The source code showed that the program:

1. Prints a message indicating that the program has started.
2. Prints a message describing harmless malware-analysis activity.
3. Creates a file named:

```
analysis_marker.txt
```

4. Writes the following text into the marker file:

```
This file was created by the benign analysis sample.
```

5. Prints messages indicating that the marker was created and that the program completed.

The inspected source code did not show functionality such as credential collection, network communication, persistence, or destructive file operations.

Figure 3: Source-code inspection of `benign_sample.py`

5.4 String Analysis

Printable strings were extracted using:

```
strings benign_sample.py
```

The output displayed the Python interpreter declaration and the messages contained in the program, including:

```
[BENIGN SAMPLE] Program started.
[BENIGN SAMPLE] Simulating harmless malware-analysis activity.
[BENIGN SAMPLE] Marker file created.
[BENIGN SAMPLE] Program completed.
```

The extracted strings were consistent with the behavior observed from the source-code inspection.

Figure 4: Printable strings extracted from the sample

5.5 File Metadata

The file metadata was inspected using:

```
ls -l benign_sample.py
```

The output showed that `benign_sample.py` was a small Python file owned by the Kali user.

The file size displayed during the analysis was:

```
344 bytes
```

This information helped establish the basic characteristics of the sample before execution.

6. Dynamic Analysis

6.1 Sandbox Preparation

A separate directory named `sandbox` was used for execution:

```
mkdir -p ~/malware-analysis/sandbox
cd ~/malware-analysis/sandbox
```

The purpose of using a separate directory was to make it easier to observe files and artifacts created by the sample.

Before execution, the directory contained no analysis-generated files.

6.2 Sample Execution

The sample was executed from the sandbox using:

```
python3 ../benign_sample.py
```

The program produced the following messages:

```
[BENIGN SAMPLE] Program started.
[BENIGN SAMPLE] Simulating harmless malware-analysis activity.
[BENIGN SAMPLE] Marker file created.
[BENIGN SAMPLE] Program completed.
```

This confirmed that the sample executed successfully and completed its programmed activity.

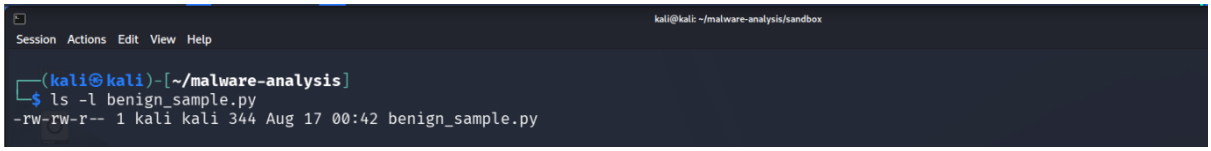

Figure 5: Execution of the benign sample inside the sandbox

6.3 Filesystem Activity

After execution, the contents of the sandbox were examined using:

```
ls -la
```

The output showed that a new file named:

```
analysis_marker.txt
```

had been created.

This demonstrated an observable filesystem change resulting from execution of the sample.

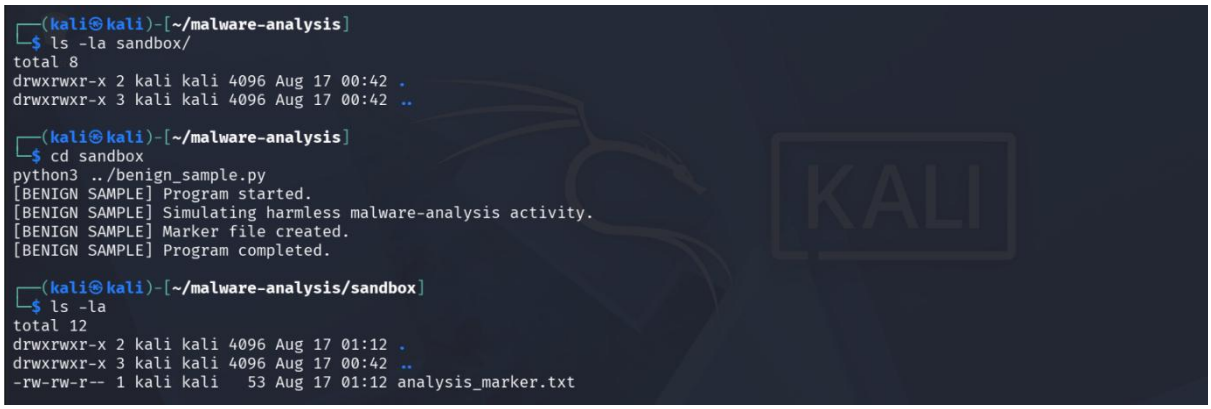

Figure 6: Verification of the file created during dynamic analysis

6.4 Verification of Created Artifact

The contents of the generated file were examined using:

```
cat analysis_marker.txt
```

The output was:

```
This file was created by the benign analysis sample.
```

This confirmed that the file creation observed during dynamic analysis matched the behavior described by the sample's source code.

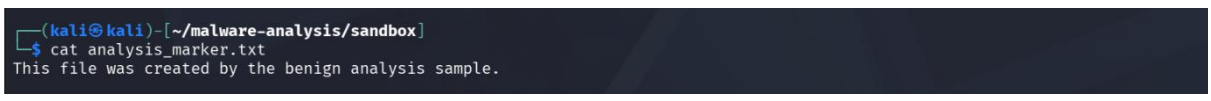

Figure 7: Verification of the contents of `analysis_marker.txt`

7. Static vs Dynamic Analysis

The exercise demonstrated the difference between the two analysis approaches.

Analysis Type	Technique Used	Observation
Static	`file`	Identified the sample as a Python script
Static	`sha256sum`	Generated a SHA-256 fingerprint
Static	`cat`	Examined the source code
Static	`strings`	Extracted printable strings
Static	`ls -l`	Examined file metadata
Dynamic	`python3`	Executed the sample
Dynamic	`ls -la`	Detected a newly created file
Dynamic	`cat`	Verified the contents of the created artifact

Static analysis provided information about the sample **without execution**, while dynamic analysis demonstrated the sample's **actual observable behavior during execution**.

8. Observed Behavior

Based on the performed analysis, the sample demonstrated the following behavior:

Before execution

- The file was identified as a Python script.
- Its SHA-256 hash was calculated.
- Its source code was inspected.
- Its printable strings were extracted.
- No execution-generated artifact was present in the sandbox.

During execution

The program:

- Printed informational messages.
- Created `analysis_marker.txt`.
- Wrote a predefined message to the file.
- Completed execution successfully.

After execution

The newly created file was identified using `ls -la` and its contents were verified using `cat`.

No additional behavior was observed during the performed analysis.

9. Security Assessment

The sample used in this exercise was **benign** and was created specifically to demonstrate malware-analysis concepts.

Based on the static and dynamic analysis performed, the sample did not demonstrate malicious behavior. Its observable activity was limited to:

- Terminal output.
- Creation of a local marker file.
- Writing predefined text to that file.

The exercise therefore served as a safe demonstration of the workflow used to examine potentially suspicious programs.

It is important to note that this conclusion applies **only to the behavior observed in the analyzed sample and the techniques performed during this exercise**. A complete malware investigation may require additional techniques such as process monitoring, network monitoring, API tracing, persistence analysis, and reverse engineering.

10. Safety Measures

The exercise used a benign sample rather than actual malware.

Dynamic execution was performed from a dedicated:

```
~/malware-analysis/sandbox
```

directory.

This helped separate the generated analysis artifact from other files in the working directory.

No actual malicious payload was introduced or executed as part of this exercise.

11. Findings Summary

Finding	Result
File type	Python script / ASCII text executable
SHA-256	Calculated successfully
Source inspection	Completed
String extraction	Completed
Dynamic execution	Completed
File creation observed	`analysis_marker.txt`
Created file verified	Successfully
Malicious behavior observed	None in the analyzed sample
Analysis environment	Kali Linux sandbox directory

12. Conclusion

The Malware Basics exercise successfully demonstrated a basic malware-analysis workflow using a benign Python sample.

During **static analysis**, the file type, SHA-256 hash, source code, printable strings, and file metadata were examined. During **dynamic analysis**, the sample was executed inside a dedicated sandbox directory and its observable filesystem activity was monitored.

The sample created `analysis_marker.txt` containing a predefined message, confirming that the observed runtime behavior matched the functionality identified during static analysis.

The exercise demonstrated how combining static and dynamic analysis can provide a better understanding of a program's characteristics and behavior while maintaining a safe analysis environment.